

DOSSIER TECHNIQUE & SCRIPT DE PRÉSENTATION

M'LAM v1.0 — FIANARANTSOA

*Système unifié de supervision urbaine, régulation du trafic et surveillance
environnementale*

« Ho an'ny tanàna milamina sy mandroso »

Pour une ville harmonieuse et prospère

Branding — L'étymologie de M'LAM

M'LAM (Mikolo's Learned App Monitoring) puise sa racine sémantique dans le terme malagasy « **Milamina** », qui désigne un état de calme, d'ordre et d'équilibre systémique. Cette philosophie structure l'ensemble de la plateforme autour de trois piliers mesurables : **l'Ordre** (fluidité de circulation), **le Calme** (stabilité environnementale) et **l'Harmonie systémique** (prospérité soutenable de la ville de Fianarantsoa).

Version 1.0 — Juillet 2026

Le Pitch de Présentation — Script Oral Minute par Minute

Ce script est conçu pour une soutenance orale de six minutes environ. Chaque bloc correspond à une phase distincte de la démonstration, avec un minutage indicatif et les points clés à exprimer à voix haute devant le jury.

0:00 - 1:00 Introduction — L'âme du projet

- **Présenter le choix de la ville de Fianarantsoa** — une ville en croissance urbaine confrontée à une densification progressive de son réseau routier.
- **Expliquer le concept fondateur « Milamina »** — l'harmonie, le calme et l'ordre comme fil conducteur de toute la conception applicative.
- **Annoncer la promesse du projet** — un outil unifié de supervision qui relie trafic routier, qualité de l'air et décision publique.

1:00 - 3:00 Le Problème & Le Modèle Hybride

- **Poser le constat** — la congestion routière sur la RN7 et les axes secondaires n'est pas qu'un problème de mobilité, c'est aussi un facteur direct de dégradation de la qualité de l'air.
- **Expliquer la chaîne de causalité** — densité véhiculaire élevée → ralentissement → stationnement prolongé → émissions accrues → accumulation de CO2 et de particules fines PM2.5 par quartier.
- **Souligner l'innovation du modèle hybride** — M'LAM ne simule pas la pollution de façon aléatoire déconnectée : elle est mathématiquement dérivée de l'état réel du trafic simulé, axe par axe, en temps réel.

3:00 - 5:00 La Démonstration du Moteur

- **Montrer la carte interactive** — React-Leaflet affiche le réseau routier réel de Fianarantsoa avec des véhicules en mouvement.
- **Basculer sur la vue Qualité de l'Air** — révéler la Heatmap de pollution en Canvas HTML5, avec ses dégradés radiaux flous représentant les nuages de pollution par quartier.
- **Déclencher ou montrer une anomalie en direct** — expliquer les causes réalistes générées par le moteur : incinération sauvage d'ordures ménagères, émissions industrielles diffuses d'ateliers de briqueterie, accumulation de gaz d'échappement des vieux moteurs Diesel et des Taxibes en l'absence de vent.

5:00 - 6:00 L'IA & La Décision Politico-Environnementale

- Cliquer sur « Générer Rapport IA » depuis une anomalie environnementale critique.
- **Expliquer la télétransmission simulée** — le rapport est automatiquement structuré et adressé, dans son en-tête, à la Direction Régionale du Ministère de l'Environnement et du Développement Durable (MEDD).
- **Insister sur la valeur ajoutée** — le rapport ne se contente pas de constater, il recommande des actions de politique publique (capteurs fixes, sensibilisation, régulation des zones industrielles).

6:00 – 7:00 Conclusion & Vision Enterprise

- **Ouvrir le panneau Paramètres** — démontrer que M'LAM n'est pas un simple prototype visuel mais une plateforme pensée pour un usage institutionnel réel.
- **Anonymisation SHA-256** — hachage cryptographique des plaques d'immatriculation des Taxibes, conformité RGPD.
- **Chiffrement AES-256** — sécurisation du canal radio et des transmissions automatiques de rapports.
- **Export CSV / PDF** — registre des anomalies exportable et bilan environnemental mensuel téléchargeable.
- **Terminer sur la devise du projet** — rappeler que M'LAM vise, au-delà de la technique, une ville plus harmonieuse et prospère pour les habitants de Fianarantsoa.

Guide de Survie Technique — Questions du Jury

2.1 — Cartographie du Code : à quoi sert chaque fichier ?

Le tableau suivant synthétise le rôle fonctionnel de chaque composant clé de l'architecture, à utiliser comme aide-mémoire en cas de question précise du jury sur l'organisation du code.

Fichier / Concept	Rôle technique
App.jsx	Le cerveau et la mémoire vive de l'application. Composant racine qui centralise tous les états globaux via useState (onglet actif, paramètres de simulation, préférences de sécurité) et orchestre le passage des données aux composants enfants par props.
CityMap.jsx	Le module de cartographie. Encapsule React-Leaflet pour afficher les tuiles géographiques, génère dynamiquement les tracés de routes et les marqueurs d'anomalies via des boucles .map() sur les données de simulation.
Canvas HTML5	Le moteur de rendu thermique. Une matrice de pixels manipulée directement via getContext('2d'), utilisée pour dessiner les véhicules en mouvement et les dégradés radiaux de pollution (createRadialGradient + filtre de flou), sans passer par le Virtual DOM de React pour préserver la fluidité à 60 images/seconde.
useEffect + Tick	Le moteur temporel de la simulation. Un hook useEffect combiné à une vérification de performance.now() joue le rôle d'un setInterval contrôlé, déclenchant toutes les 2,5 secondes la génération d'anomalies aléatoires (Math.random()) et l'ajustement des indices de pollution par axe routier.
useState	La mémoire vive réactive. Chaque changement d'état (filtre actif, thème, seuils d'alerte) déclenche un nouveau rendu ciblé de l'interface, garantissant une synchronisation immédiate entre les paramètres et l'affichage.
useRef	La mémoire persistante hors-rendu. Utilisé pour stocker la flotte de véhicules simulée sans déclencher de re-rendu React à chaque frame, un choix essentiel

2.2 — Explication pas-à-pas des choix technologiques

Pourquoi React ?

React impose une architecture par composants réutilisables et un modèle de données unidirectionnel (state descendant vers les props). Pour une application de supervision où de nombreux indicateurs doivent rester synchronisés en permanence (carte, sidebar, compteurs, modale de rapport), ce modèle réduit drastiquement les risques d'incohérence entre les vues, comparé à une manipulation manuelle du DOM.

Pourquoi Vite ?

Vite exploite les modules ECMAScript natifs du navigateur et compile à la demande, ce qui donne un démarrage quasi instantané du serveur de développement et un Hot Module Replacement préservant l'état de la simulation en cours pendant le développement — un gain de temps déterminant sur un projet itératif comme celui-ci.

Pourquoi Tailwind CSS ?

Tailwind permet de construire une charte graphique cohérente — le style « Blueprint / Carreau mat » de M'LAM — directement dans le balisage, sans jongler entre des dizaines de fichiers CSS. Cette approche garantit une homogénéité visuelle stricte entre tous les écrans de l'application, essentielle pour un outil de supervision professionnel où la lisibilité prime sur la décoration.

Quelle différence entre HTML Canvas et HTML5 ?

HTML5 est la version du langage de balisage qui a introduit, entre autres nouveautés, l'élément `<canvas>`. Le Canvas n'est donc pas un concept concurrent de HTML5 mais l'une de ses briques constitutives : une zone de dessin bitmap manipulable via JavaScript (API Canvas 2D), utilisée dans M'LAM pour tout rendu graphique à haute fréquence (véhicules, nuages de pollution) que le DOM classique ne pourrait pas gérer sans perte de performance.

Pourquoi React-Leaflet est une bibliothèque et non un framework ?

React-Leaflet est une bibliothèque car elle fournit un ensemble ciblé de composants React (TileLayer, Marker, Polyline) qui encapsulent Leaflet.js, sans imposer de structure globale à l'application. Contrairement à un framework, qui dicterait l'architecture complète du projet, une bibliothèque s'intègre localement là où elle est nécessaire — ici, uniquement dans le composant CityMap.jsx — laissant React piloter le reste de l'application.

Fiche Résumée — En Cas de Bug sur Scène

Une démonstration en direct comporte toujours un risque technique. Voici les trois réflexes à appliquer dans l'ordre, sans perdre le fil de la présentation.

PROTOCOLE D'URGENCE — 3 CONSIGNES

1. Relancer le serveur de développement avec la commande : `npm run dev -- --force` — cette option force Vite à reconstruire le cache de dépendances, résolvant la majorité des erreurs d'affichage inexplicables.
2. Ouvrir l'application dans une fenêtre de navigation privée — cela élimine instantanément les conflits liés au cache navigateur ou aux extensions installées, cause fréquente de bugs visuels fantômes.
3. Garder le calme et commenter à voix haute — un jury valorise davantage la capacité à diagnostiquer calmement un incident en direct que l'absence totale d'imprévu. Décrire ce qui se passe pendant la résolution transforme l'incident en démonstration de maîtrise technique.

« *Ho an'ny tanàna milamina sy mandroso* »